



MODUL 13 · UPLOAD FILE & OPTIMASI GAMBAR

Upload ke Cloud Storage: Cloudinary

Storage + CDN + transformasi on-the-fly. Free tier 25GB. Production-ready dari hari pertama.



U P L O A D F I L E & N E X T / I M A G E

Kenapa Cloud Storage Wajib untuk Production?

Local storage punya 5 masalah besar. Cloud storage menyelesaikan semuanya.



Vercel filesystem tidak persistent



Cloud: file tidak hilang setelah deployment



Serverless bisa spawn banyak instance



Semua instance akses satu cloud storage



Butuh CDN untuk performa global



Cloudinary punya CDN built-in di 100+ PoP



Perlu resize/compress gambar



Cloudinary transform on-the-fly via URL parameter



Backup dan durability



Cloudinary 99.9% uptime, multi-region backup

Cloudinary free tier: 25GB storage + 25GB bandwidth/bulan. Cukup untuk seluruh kursus dan portfolio. UploadThing adalah alternatif lebih Next.js-native jika prefer simpler API.

U P L O A D F I L E & N E X T / I M A G E

Setup Cloudinary — lib/cloudinary.ts + Upload Action

bun add cloudinary. Konfigurasi sekali, upload via base64 data URI dari Server Action.

```
import { v2 as cloudinary } from "cloudinary"

cloudinary.config({
  cloud_name:
  process.env.CLOUDINARY_CLOUD_NAME!,
  api_key:
  process.env.CLOUDINARY_API_KEY!,
  api_secret:
  process.env.CLOUDINARY_API_SECRET!,
})

export { cloudinary }

// .env:
// CLOUDINARY_CLOUD_NAME=your_name
// CLOUDINARY_API_KEY=123456789
// CLOUDINARY_API_SECRET=abc123 ...

// ⚠️ JANGAN prefix NEXT_PUBLIC!
// API secret hanya di server.

// Dari dashboard.cloudinary.com:
// Settings → API Keys
```

```
"use server"
import { cloudinary } from "@lib/cloudinary"
import { nanoid } from "nanoid"

export async function uploadToCloud(
  formData: FormData
) {
  const file = formData.get("file") as File | null
  if (!file || file.size === 0)
    return { error: "File tidak ada" }

  // File → Buffer → Base64 → Data URI
  const bytes = await file.arrayBuffer()
  const buffer = Buffer.from(bytes)
  const base64 = buffer.toString("base64")
  const dataUri =
`data:${file.type};base64,${base64}`

  // Upload ke Cloudinary
  const result = await cloudinary.uploader.upload(
    dataUri,
    {
      folder: "blog-covers",
      public_id: nanoid(),
      overwrite: false,
      transformation: [
        { width: 1200, height: 630, crop: "fill" },
        { quality: "auto" }, // compress optimal
        { fetch_format: "auto" }, // WebP/AVIF
      ],
    }
  )

  return { url: result.secure_url }
}
```

U P L O A D F I L E & N E X T / I M A G E

Pattern Avatar Upload — Full Flow

Auth → Validasi → Upload Cloudinary → Simpan URL ke DB → Revalidate. Siklus lengkap.

```
"use server"
import { cloudinary } from "@lib/cloudinary"
import { db } from "@lib/db"
import { auth } from "@lib/auth"
import { headers } from "next/headers"
import { revalidatePath } from "next/cache"

export async function updateAvatar(formData: FormData) {
  // 1. Auth
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) return { error: "Unauthorized" }

  const file = formData.get("avatar") as File | null
  if (!file || file.size === 0) return { error: "File diperlukan" }

  // 2. Validasi (lengkap di Bab 3 dengan Zod!)
  const TYPES = ["image/jpeg", "image/png", "image/webp"]
  const MAX = 2 * 1024 * 1024
  if (!TYPES.includes(file.type)) return { error: "Hanya JPEG/PNG/WebP" }
  if (file.size > MAX) return { error: "Maks 2MB" }

  // 3. Upload — public_id pakai userId agar overwrite foto lama
  const b64 = Buffer.from(await file.arrayBuffer()).toString("base64")
  const result = await cloudinary.uploader.upload(
    `data:${file.type};base64,${b64}`,
    { folder: "avatars", public_id: `user-${session.user.id}`,
      overwrite: true, transformation: [{ width: 200, height: 200, crop: "fill" }] }
  )

  // 4. Simpan URL ke DB
  await db.user.update({ where: { id: session.user.id }, data: { image: result.secure_url } })

  // 5. Revalidate
  revalidatePath(`/profile/${session.user.id}`)
  return { url: result.secure_url, success: true }
}
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa API secret Cloudinary TIDAK boleh diprefix NEXT_PUBLIC_?

- A) Akan error saat build
- B) NEXT_PUBLIC_ expose ke browser bundle — API secret hanya boleh di server
- C) Cloudinary tidak terima public vars

Q2 — Bagaimana cara mengupload File ke Cloudinary dari Server Action?

- A) Pass file.path langsung
- B) Convert ke base64 data URI → `cloudinary.uploader.upload(dataUri, options)`
- C) Pakai `fetch()` ke Cloudinary API

Q3 — Setelah `updateAvatar` berhasil dan URL disimpan ke DB, apa yang perlu dilakukan?

- A) Tidak perlu apa-apa
- B) `revalidatePath` atau `revalidateTag` agar Server Components yang tampil avatar ikut refresh
- C) `window.location.reload()`

Tugas Mandiri

Implementasikan upload avatar ke Cloudinary.

01 Cloudinary Setup

Buat akun cloudinary.com (free). Copy credentials ke .env. bun add cloudinary. Buat lib/cloudinary.ts dengan cloudinary.config().

02 Upload Helper

Buat fungsi uploadImage(file: File): Promise<{url: string}> di lib/cloudinary.ts yang encapsulate convert + upload logic.

03 Avatar Action

Buat updateAvatar Server Action: auth → validasi tipe+ukuran → upload folder 'avatars' dengan overwrite:true → update DB → revalidatePath.

04 Preview Form

Buat AvatarUploadForm: file input + preview (URL.createObjectURL) + submit + tampilkan hasil upload dari server.



next.config.ts: tambahkan res.cloudinary.com ke images.remotePatterns agar next/image bisa load URL Cloudinary.

Yang Sudah Kita Pelajari

- ✓ Cloud storage wajib production: Vercel filesystem tidak persistent.
- ✓ `bun add clouinary. lib/clouinary.ts`: konfigurasi sekali. Kredensial JANGAN NEXT_PUBLIC_!
- ✓ Upload: File → `arrayBuffer` → `base64 data URI` → `clouinary.uploader.upload()`.
- ✓ `public_id` unik + `overwrite:true` untuk avatar — replace gambar lama otomatis.
- ✓ Setelah update DB → `revalidatePath/Tag` agar Server Components ikut refresh.

→ Selanjutnya: **Bab 3 — Validasi File dengan Zod**

